



Bônus — Orientação a Objetos em ADVPL/Harbour

Jornada DEV START · Material bônus de autoestudo · Programa START — TOTVS

Paulista Sem entrega, sem nota, sem prazo. É pra você estudar no seu ritmo, com suporte no Discord em [#duvidas-partel1-harbour](#) até **07/08**. **Roda em Harbour puro** — todo exemplo deste material usa fence `harbour` e compila com `hbm2`, sem precisar de ambiente Protheus. Se você não chegou a montar o AppServer, não tem problema: dá pra fazer 100% deste bônus só com o que já instalou no pré-curso.

Por que isso importa (mesmo tendo perdido a aula ao vivo)

Este material consolida o conteúdo do **Módulo 10 — Orientação a Objetos**, a última aula de conteúdo novo da jornada. Se o seu curso terminou antes de chegar aqui, ou se você quer revisar com calma, este bônus cobre exatamente o que seria dado em aula: `CLASS`, `DATA`, `METHOD`, `New()`, encapsulamento e herança — junto com os exercícios do módulo, já com gabarito comentado.

Vale reforçar: você **não precisa** disso para nada obrigatório do curso. É conteúdo a mais, pra quem quer entender um paradigma novo e poderoso, e que também abre a porta para entender classes do próprio Protheus (`FWMsgRun`, `TReport`, etc.) quando você chegar lá.

1. Por que Orientação a Objetos?

Até aqui você programou de forma **estruturada**: funções que manipulam dados soltos. Funciona muito bem, e vai continuar sendo a maior parte do seu código. Mas para sistemas maiores, a **Orientação a Objetos (POO)** organiza melhor as coisas, juntando dados e comportamento num só lugar.

Programação Estruturada	Orientação a Objetos
Dados e funções separados	Dados + comportamento juntos
Qualquer função acessa qualquer dado	Acesso controlado
Reuso difícil	Herança e composição
Difícil escalar	Modular por natureza

💡 POO **não substitui** a programação estruturada — ela **complementa**, para os casos em que juntar “dados + comportamento” num só lugar deixa tudo mais organizado.

2. Os cinco conceitos fundamentais

Classe — é o *molde*, a planta baixa. Define quais atributos e métodos os objetos terão.

```
Classe Contato:
  Atributos (dados):   Codigo, Cliente, Assunto, Data
  Métodos (ações):     Validar(), Salvar(), ToString()
```

Objeto (instância) — é o *exemplar* criado a partir do molde. De uma classe saem vários objetos:

```
oContato1 := Contato():New("000001", "000001", "Reunião")
oContato2 := Contato():New("000002", "000002", "Suporte")
```

- **Encapsulamento** — esconder os detalhes internos; expor só o necessário.
- **Herança** — uma classe reaproveita atributos e métodos de outra.
- **Polimorfismo** — o mesmo método (`Validar()`), por exemplo) se comporta diferente em classes diferentes.

3. Criando uma classe em Harbour

A estrutura tem **duas partes**: a **definição** (`CLASS ... ENDCLASS` , o “índice” da classe) e a **implementação** de cada método, separada:

```
// ---- Definição da classe ----
CLASS Contato

    DATA cCodigo    AS CHARACTER    // atributos (dados)
    DATA cCliente   AS CHARACTER
    DATA cAssunto   AS CHARACTER
    DATA dData      AS DATE

    METHOD New(cCod, cCli, cAss)      // construtor
    METHOD Validar()                 // métodos (ações)
    METHOD ToString()

ENDCLASS

// ---- Implementação dos métodos ----
METHOD New(cCod, cCli, cAss) CLASS Contato
    ::cCodigo := cCod
    ::cCliente := cCli
    ::cAssunto := cAss
    ::dData    := Date()
RETURN SELF    // retorna o próprio objeto

METHOD Validar() CLASS Contato
RETURN !Empty(::cCliente) .AND. !Empty(::cAssunto)

METHOD ToString() CLASS Contato
RETURN ::cCodigo + " | " + ::cCliente + " | " + ::cAssunto

FUNCTION Main()
    LOCAL oContato := Contato():New("000001", "000001", "Reunião inicial")

    QQout(oContato:ToString())

    IF oContato:Validar()
        QQout("Dados válidos!")
    ENDIF

    QQout("Data: " + DToC(oContato:dData))
RETURN NIL
```

Compile e rode com:

```
hbmK2 contato.prg
contato.exe
```

Dois símbolos que você precisa decorar

Símbolo	Significado
<code>::</code>	“deste objeto” — <code>::cCodigo</code> é o atributo <code>cCodigo</code> deste objeto
<code>SELF</code>	o próprio objeto — o construtor sempre retorna <code>SELF</code>

Repare no padrão: `Classe():New(...)` para **criar** o objeto, e `objeto:Metodo()` para **usar** o objeto. É esse mesmo padrão que aparece em qualquer classe do Protheus.

4. ACCESS e ASSIGN — encapsulamento na prática

Expor os atributos “crus” (`DATA`) deixa qualquer um mudar qualquer coisa, sem controle. Com `ACCESS` (getter, leitura) e `ASSIGN` (setter, escrita) você **controla** o acesso — e pode **validar na escrita**:

```
CLASS Produto

    DATA _nPreco AS NUMERIC    // "privado" (convenção: prefixo _)

    METHOD New(nPre)

        ACCESS nPreco        INLINE ::_nPreco                // leitura
        ASSIGN nPreco(n)     INLINE IF(n >= 0, ::_nPreco := n, NIL) // valida ao escrever

    ENDClass

    METHOD New(nPre) CLASS Produto
        ::nPreco := nPre    // passa pelo ASSIGN → só aceita >= 0
    RETURN SELF

    FUNCTION Main()
        LOCAL oProd := Produto():New(29.90)

        oProd:nPreco := -5    // o ASSIGN bloqueia (preço negativo não entra)
        QOut(oProd:nPreco)    // continua 29.90
    RETURN NIL
```

O objeto **protege a si mesmo**: por mais que o código de fora tente, um preço negativo nunca entra. Isso é encapsulamento — a classe decide o que é um valor válido para ela mesma.

5. Herança — reaproveitando código

Uma classe pode **herdar** tudo de outra (**FROM**) e adicionar ou **sobrescrever** comportamentos:

```

// ---- Classe base ----
CLASS Cadastro

    DATA cCodigo    AS CHARACTER
    DATA lExcluido  AS LOGICAL

    METHOD New()
    METHOD Ativo()

ENDCLASS

METHOD New() CLASS Cadastro
    ::lExcluido := .F.
RETURN SELF

METHOD Ativo() CLASS Cadastro
RETURN !::lExcluido

// ---- Classe derivada: herda de Cadastro ----
CLASS Fornecedor FROM Cadastro

    DATA cNome    AS CHARACTER
    DATA cCNPJ    AS CHARACTER

    METHOD New(cCod, cNome, cCNPJ)

ENDCLASS

METHOD New(cCod, cNome, cCNPJ) CLASS Fornecedor
    ::Super:New()    // chama o construtor da classe PAI
    ::cCodigo := cCod
    ::cNome   := cNome
    ::cCNPJ   := cCNPJ
RETURN SELF

FUNCTION Main()
    LOCAL oFornec := Fornecedor():New("001", "ACME Corp", "12345678000190")

    IF oFornec:Ativo()    // método HERDADO de Cadastro!
        QOut("Fornecedor ativo!")
    ENDIF
RETURN NIL

```

`::Super:New()` é a chave da herança: a classe filha chama o construtor do **pai** antes de configurar o que é só dela. Assim nada é duplicado.

6. Quando usar POO?

Situação	Use POO?
CRUD simples (AxCadastro/mBrowse)	❌ Desnecessário
Processo com muitos estados	✅
Componente reutilizável em vários módulos	✅
Hierarquia de entidades (Produto → Físico/Digital)	✅
Bibliotecas do Protheus (<code>FWMsgRun</code> , <code>TReport</code> ...)	✅ Necessário

Quando você chegar (ou voltar) ao Protheus, vai notar que várias ferramentas prontas já são orientadas a objetos: `FWMsgRun():New(...)` , `TReport():New(...)` . Entender POO agora é o que permite ler o código dessas bibliotecas sem se assustar.

7. Exercícios opcionais (com gabarito comentado)

Os exercícios abaixo são os mesmos do Módulo 10. Tente fazer sozinho primeiro — o gabarito está comentado logo depois de cada enunciado, então cubra a tela se quiser se testar de verdade. 😊

Exercício 1 — Classe `Calculadora`

Crie uma classe `Calculadora` que permita **encadear** operações:

- Atributo `nAcumulador` (inicia em `0`)
- Métodos `Somar(n)` , `Subtrair(n)` e `Resultado()`
- Cada operação retorna `SELF` (para poder encadear)

Uso esperado: `oCalc:Somar(10):Somar(5):Subtrair(3)` e depois `oCalc:Resultado()` → **12**.

►  Gabarito comentado — Exercício 1

Exercício 2 — Classe `Contato` com encapsulamento

Crie uma classe `Contato` com:

- Atributos `cCodigo` , `cCliente` , `cAssunto` , `dData`

- Construtor `New()` que preenche `dData` com `Date()` automaticamente
- Método `Validar()` que retorna `.T.` só se `cCliente` e `cAssunto` estiverem preenchidos
- `ACCESS` / `ASSIGN` para `cAssunto` , limitando a **60 caracteres** na escrita

►  Gabarito comentado — Exercício 2

Exercício 3 — Desafio (opcional): herança

Crie a hierarquia de classes:

- `Pessoa` (atributos `cNome` , `cCPF` , `dNasc`): métodos `Validar()` e `Idade()`
- `PessoaFisica` `FROM Pessoa` (atributo `cRG`): **sobrescreve** `Validar()` adicionando a checagem do CPF
- `PessoaJuridica` `FROM Pessoa` (atributos `cCNPJ` , `cRazaoSocial`): **sobrescreve** `Validar()` adicionando a checagem do CNPJ

►  Gabarito comentado — Exercício 3



Referência rápida

```
// --- Definir uma classe ---  
CLASS Contato  
    DATA cCodigo AS CHARACTER           // atributo  
    METHOD New(cCod)                      // método  
    ACCESS nX INLINE ::_nX              // getter  
    ASSIGN nX(v) INLINE ::_nX := v      // setter  
ENDCLASS  
  
// --- Implementar um método ---  
METHOD New(cCod) CLASS Contato  
    ::cCodigo := cCod                   // :: = "deste objeto"  
RETURN SELF                            // construtor retorna SELF  
  
// --- Herança ---  
CLASS Fornecedor FROM Cadastro ... ENDCLASS  
::Super:New()                         // chama o construtor do pai  
::Super:Metodo()                      // chama a versão do pai de um método  
sobrescrito  
  
// --- Usar ---  
oObj := Contato():New("001")           // instanciar  
oObj:Metodo()                          // chamar método  
oObj:cCodigo                           // acessar atributo
```

Elemento	Para que serve
<code>CLASS ... ENDCLASS</code>	Define o molde (atributos + assinatura dos métodos)
<code>DATA</code>	Atributo público (dado guardado no objeto)
<code>METHOD ... CLASS X</code>	Implementação de um método da classe X
<code>New()</code>	Construtor — sempre RETURN SELF no final
<code>::</code>	Acessa algo deste objeto (dentro de um método)
<code>SELF</code>	O próprio objeto
<code>ACCESS</code> / <code>ASSIGN</code>	Getter / setter — encapsulamento com validação
<code>FROM</code>	Declara herança de outra classe
<code>::Super:Metodo()</code>	Chama a versão do método na classe pai

🎯 Se você fez o Exercício 1 e o 2, já fixou o essencial de OOP. O Exercício 3 é só para quem quer ir além — sem problema nenhum deixar pra depois. Dúvidas: [#duvidas-parte1-harbour](#). 🔍